Retrieval Augmented Structured Generation: Business Document Information Extraction As Tool Use

Franz Louis Cesista
Expedock Software, Inc
San Francisco, United States
franzlouiscesista@gmail.com

Rui Aguiar
Expedock Software, Inc
San Francisco, United States
rui@expedock.com

Jason Kim
Expedock Software, Inc
San Francisco, United States
jasonminsookim@gmail.com

Paolo Acilo
Expedock Software, Inc
San Francisco, United States
paolo@expedock.com

***Abstract*—Business Document Information Extraction (BDIE) is the problem of transforming a blob of unstructured information (raw text, scanned documents, etc.) into a structured format that downstream systems can parse and use. It has two main tasks: Key-Information Extraction (KIE) and Line Items Recognition (LIR). And subtasks such as Optical Character Recognition (OCR) and Table Structure Recognition (TSR) are means to these ends. In this paper, we argue that BDIE is best modeled as a *Tool Use* problem, where the tools are these downstream systems. We then present Retrieval Augmented Structured Generation (RASG), a novel general framework for BDIE that achieves state of the art (SOTA) results on both KIE and LIR tasks on BDIE benchmarks.**

The contributions of this paper are threefold: (1) We show, with ablation benchmarks, that Large Language Models (LLMs) with RASG are already competitive with or surpasses current SOTA Large Multi-Modal Models (LMMMs) without RASG such as LayoutLMv3 and Roberta + DeTR on BDIE benchmarks. (2) We propose a new metric class for Line Items Recognition, General Line Items Recognition Metric (GLIRM), that is more aligned with practical BDIE use cases compared to existing metrics, such as ANLS*, DocILE, and GriTS. (3) We provide a heuristic algorithm for backcalculating bounding boxes - that is, pairs of (x, y) coordinates containing relevant text of predicted line items and tables without the need for vision encoders. Finally, we claim that, while LMMMs might sometimes offer marginal performance benefits, LLMs + RASG is oftentimes superior given real-world applications and constraints of BDIE.

***Index Terms*—Document Information Extraction, Key Information Extraction, Line Items Recognition, Retrieval Augmented Generation, Structured Generation**

I. Introduction

In practice, organizations or individuals do Business Document Information Extraction (BDIE) because they want to connect human organizations and systems such as businesses and governments with indefinite interfaces (such as printed/handwritten documents) to computer programs with definite interfaces (such as Application Program Interfaces). These programs then process the inputs and save the results to databases before presenting them to end users. Thus, for a machine learning system to be good at BDIE, then it has to be good at interfacing with these downstream programs, as well as at least minimally aware of what these programs do with the data. In the succeeding sections, we will discuss how we can achieve these two goals with our new methodology, Retrieval Augmented Structured Generation (RASG), and our new proposed metric class, General Line Items Recognition Metric (GLIRM).

Document Information Extraction is a vast field that encompasses tasks such as, but is not limited to, Optical Character Recognition (OCR) [1], Table Structure Recognition (TSR) [2] [3], Document Retrieval [4], Document Vision Question-Answering (DocVQA) [5], Key-Information Extraction (KIE) [6], and Line Items Recognition (LIR) [7].

In OCR and TSR, the goal is to extract text and tables from document images, respectively. In document retrieval, the goal is to find the most relevant documents given a query. While many providers already offer off-the-shelf solutions for these tasks, their outputs are still semi-structured and require further processing before we could apply business logic to them. DocVQA, on the other hand, is the task of answering user questions about the content of the document images. It can be modeled as a combination of the previous tasks, but it is more concerned with interfacing with users rather than with downstream systems.

The goal of KIE is to extract and format information from the document into key-value pairs. Likewise, the goal of LIR is to extract information into line items where each line item corresponds to a row in a table formatted into column key-value pairs. The keys being predefined is important because we use them to apply business logic in downstream systems. For example, Invoice A could contain a field called `"Total Amount"` and Invoice B could contain a field called `"Total Amount Due"`. These fields should map to the same key so that a downstream system could process them in the same way. Thus, in this paper, we focus on the KIE and LIR tasks.

KIE has a healthy research environment. There are many publicly-available datasets for it such as FUNSD [8], CORD [9], and SROIE [10]. The metrics for it are also simple and well-defined. The Average Precision and F1 generally suffice for performance benchmarking. LIR, on the other hand, is still largely overlooked as noted by [7]. The only publicly-available, large-enough dataset specifically constructed for it is the DocILE dataset [11] [12]. The evaluation metrics for LIR are also incomplete, which we discuss later.

We also aim to make this paper as useful to practitioners as possible. Many organizations that do BDIE do not have the budget to train LLMs from scratch. Thus, we focus on augmenting off-the-shelf commercially-available and/or open-

source LLMs with RASG.

One problem often encountered in practical applications of open-source models is that the only cost-effective models often have a small token limit or are not as powerful. This leads to the question: "What is the minimal amount of resources we need to achieve SOTA results on BDIE benchmarks?" Later in this paper, we will answer this question by providing ablation benchmarks on the DocILE dataset.

There have also been several attempts to combine text, layout, and visual and textual information for BDIE such as LayoutLMv3 [13] and DocLLM [14]. LayoutLMv3 combines all three modalities additively. DocLLM, on the other hand, only combines visual and textual information, but through cross-attention and yet it outperforms even GPT-4V on DocVQA benchmarks. If visual information is not so important at all on BDIE-related tasks, then perhaps we might not even need the layout information if clever prompt engineering is used [1].

There are also several attempts to embed visual and layout information into the text prompt such as Language-Interfaced FineTuning (LIFT) and LAyout Task-aware INstruction Prompt (LATIN-prompt) [15] [16]. These papers show that proper prompt engineering can be a good alternative to using large multi-modal models. This is especially important for commercial applications because it enables businesses to use off-the-shelf LLMs without the need for expensive pretraining or finetuning.

To summarize and further motivate RASG: the goal of BDIE is to transform a blob of information into a structured format we can pass to downstream tools. We can teach machine learning models to use tools through supervised finetuning. We can then force them to output in the format we expect using structured generation. We can also teach sufficiently-powerful, pre-trained models to use new tools on out-of-distribution datasets by taking advantage of in-context learning. And finally, to be able to use commercial LLMs "out-of-the-box", we can structure the text prompt to look like the original document through prompt engineering.

II. Background

Document information extraction refers to the process of automatically locating, extracting, and structuring specific information from document based sources, such as invoices or receipts. Over the years, AI has played a crucial role in revolutionizing this field. Early AI-based information extraction relied on procedural-based approaches to information extraction [17] [18]. These systems employed hand-crafted rules including regular expression matching and conditional logic that identified patterns and keywords within documents to extract specific data points. While effective for well-defined tasks and structured documents, rule-based systems struggled with flexibility and scalability. They are labor-intensive to maintain, with the largest issue being the requirement of continuous rule updates for handling variations in language and document formats.

In more recent years, statistical approaches emerged as an alternative for document information extraction, leveraging machine learning and deep learning techniques [17]. These methods involved using large annotated document and language sets which statistical models were applied to to effectively perform information extraction. The models learned to identify patterns and relationships between textual features and desired information. Techniques here included part-of-speech (POS) taggers [19] which could accept textual input and output the grammatical structure of a document. Using tooling like the POS tagger, deep learning has extensively been applied to the IE problem [6]. Long-Short term memory networks have been applied to IE using part of speech tagging as input. The recent emergence of Large Language Models (LLMs) has significantly impacted the field of document information extraction [20]. Approaches using LLMs rely on a pipeline of performing optical character recognition to convert the document into a textual input, chunking the input, prompt generation and applying LLM-based inference to the chunked prompt in the hope that the LLM completions can decode the document. Results here have been promising, though this field of application is still new.

A. Table Structure Recognition vs. Line Items Recognition

Here, we make a distinction between the Table Structure Recognition (TSR) task and the Line Items Recognition (LIR) task. In TSR, we expect to receive a grid of cells (which may span multiple rows or columns) where the order of the columns are preserved. In LIR, the order of the columns does not matter so long as the columns are mapped to the proper, predefined column keys.

These tasks share two subtasks:

1) **Cell Content Recognition**: where we want to maximize the accuracy of the text content of the cells
2) **Cell Location Recognition**: where we want to maximize the accuracy of the absolute coordinates of the cells.

The Grid Table Structure (GriTS) metric [21] is appropriate for TSR and can handle both subtasks, but it is too strict on the grid structure to be used for LIR.

Other proposed metrics for LIR such as ANLS* and DocILE, on the other hand, can only handle cell content recognition and does not penalize "shuffled" rows - for example, predicting a row in the middle of the table as the first row in the result [22]. This is because these metrics match predicted line items to ground truth line items using Maximum-Weight Bipartite Matching-based algorithms which do not preserve the order of the line items [23]. After, we introduce a new metric that can handle both subtasks and properly penalizes shuffled rows in the predictions.

B. In-Context Learning and Retrieval Augmented Generation

Retrieval augmented language models condition a foundation model on relevant data from a grounding corpus during generation, which can significantly improve performance [24].

[1]This idea was inspired by a bug we encountered in production which fed noisy image (and potentially bounding box) information to our model. The model still performed well, albeit with a slight drop in performance.

Retrieval augmented generation (RAG) is an extension of this, where a general purpose fine-tuning using a non-parametric retriever in addition to a query encoder assists a language model in question answering [25]. In BDIE, there is the natural application of RAG where one retrieves similar documents by some metric to assist in information extraction.

C. Tool Use and Structured Generation

There have recently been numerous experiments run concerning the ability of foundation models to learn the usage of tools. We will refer to the ability of a foundation model's ability to teach themselves to use external tools via simple APIs as *Tool Use*. One approach, such as ToolFormer [26], is to finetune a foundation model to interface with APIs when responding to a prompt. Another approach is to constrain the outputs of a frozen model *post-training*. Some such as Instructor [27] impose "soft" constraints where they request the LLM to produce output following a certain schema and retry if they fail. Others such as Outlines [28] impose harder constraints where they zero-out probabilities of invalid tokens.

D. Structured Prompting

When performing BDIE using a generative language model, structured output is essential for correctness. It is oftentimes the case that in a structured prediction problem such as named entity recognition, language models do not perform in a robustly generalized manner without explicit prompting [29]. Careful prompt engineering, as well as in-context learning, can correct issues in language model output [30]. This has even been explored in the context of zero-shot document question answering explicitly, where layout-aware document and question structuring has been shown to improve question answering performance in zero-shot prompting environments [16].

III. Retrieval Augmented Structured Generation (RASG)

RASG is composed of four components: **(1) Retrieval Augmented Generation** allows us to teach LLMs to use new tools through examples using In-Context Learning; **(2) Supervised Finetuning** enhances the correctness of the extracted outputs; **(3) Structured Generation** ensures that the outputs are parseable by downstream programs; and **(4) Structured Prompting** infuses layout information into the prompt which then allows us to use widely-available LLMs.

All four components are necessary to beat strong multimodal baselines on BDIE using an open-source 7B LLM, Hermes 2 Pro - Mistral 7B [31]. But only a subset are necessary when using GPT-3.5 [32].

A. Finetuning for Structured Generation

The language model used needs to output both the right *content* and the right *structure* of said content. Finetuning significantly helps with the former, but only to a certain extent with the latter. To ensure that the output is parseable by downstream systems, we need to zero-out the probabilities of invalid tokens. This is where structured generation comes in.

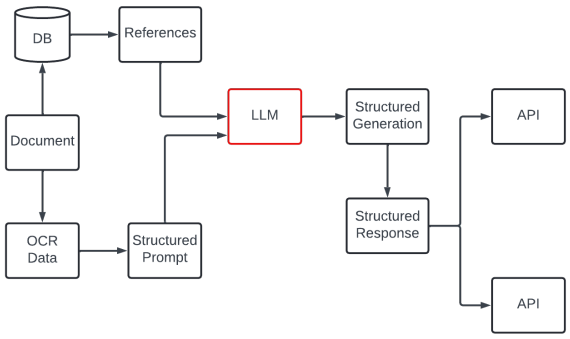

Fig. 1. Retrieval Augmented Structured Generation (RASG)

Based on our experiments, however, we found that naively combining finetuning and structured generation sometimes leads to even worse results. This was first noted by [33]. There are two issues we need to be wary of and we will address them in turn. Said issues are:

1. **Schema vs. model mismatch**: Finite State Machine-based algorithms for structured generation, such as Outlines' `outlines.generate.fsm` module, implicitly impose a strict key ordering. That is, if the key `"amount"` comes before `"currency"` in the schema, then Outlines will zero-out the probabilities for the tokens corresponding to `"currency"` until `"amount"` is predicted. However, if the model was finetuned to generate `"currency"` before `"amount"`, then the prediction accuracy would collapse. To remedy this, one must either ensure that the dataset strictly follows the specified schema, or use Context-Free Grammar-based algorithms for structured generation, such as Outlines' `outlines.generate.cfg` module, which does not impose a strict key ordering.

2. **Token explosion with optional keys**. A common issue we have observed is to require the keys to be generated even when the predicted value is `null`. This happens, for example, when one builds a Pydantic object with `Optional` fields then pipes the object's json schema to Outlines. This leads to a lot of unnecessary tokens being generated, slowing down inference. Another bad practice when using Outlines is to make *all* of the keys optional. This is because Outlines uses a different algorithm to generate the FSM for this edge case. A workaround for this is to add a required dummy key of type `null` to the schema and simply remove it in the postprocessing step.

B. Bounding Box Backcalculation Heuristic

For the KIE task, we have found that a simple, greedy heuristic (Algorithm 1) is sufficient for backcalculating bounding boxes in practice. To use the entire page, simply set the y lowerbound and upperbound to 0 and the page height in pixels, respectively. For the LIR task, a good heuristic is to (1) divide the page vertically into chunks, one for each line item; and (2) re-use Algorithm 1 above to back-calculate the bounding boxes for each line item, but only for the words in

the chunk assigned to the line item. The challenge then is *how* to divide the page into chunks.

inputs: y lowerbound and upperbound, predicted key-value map, and OCR Data
output: Matching score, key to bounding box map

$score \leftarrow 0; key_bbox_map \leftarrow \{\}$;
foreach $(key, value)$ *pair in the key-value map* **do**
  $m_words \leftarrow$ The longest contiguous list of words matching $value$ whose bounding boxes lie within the y lowerbound and upperbound;
  $m_bboxes \leftarrow$ Bounding boxes of m_words;
  $key_bbox_map[key] \leftarrow union(m_bboxes)$;
  $score \leftarrow score + similarity_measure(concat(m_words)), len(value))$
end
return $(score, key_bbox_map)$;

Algorithm 1: Heuristic for backcalculating bounding boxes for key-value maps

The naive dynamic programming approach with 2D states `(line item index, page` y`)` has complexity $O(M * N^2 * O(\text{Algo 1}))$ where M is the number of line items and N is the page height. We can optimize this by (implicitly) down-scaling the page first. In production we use $N = 128$. A further optimization takes advantage of the monotonicity of Algorithm 1: the matching score is non-increasing as we increase the y lower bound and/or decrease the y upper bound. Therefore, we can use the Divide-and-Conquer optimization for dynamic programming problems [34] [35] to speed this up to $O(M * N \log N * O(\text{Algo 1}))$. Finally, we employ binary search to find the largest y lowerbound for the first line item and the smallest y upperbound for the last line item to further tighten the bounds.

IV. General Line Items Recognition Metric (GLIRM)

In this section, we will derive a new metric class for Line Items Recognition (LIR) from first principles. First, we will discuss the attributes an ideal metric for LIR should have. Then, we will discuss the metric itself.

The ideal metric for LIR should have the following attributes:

1) **Subtask Isolation**: Performance on the subtasks must be measured separately.
2) **Cell Isolation**: A True Positive corresponds to exactly one predicted cell and one ground truth cell.
3) **Cell Completeness**: Hallucinated cells are counted as False Positives. And missing cells are counted as False Negatives.
4) **Cell Similarity-Measure Flexibility**: Within the same subtask, the metric must be able to support multiple cell similarity measures.
5) **Cell Row-Position Invariance**: The credit given to a correctly predicted cell is the same regardless of its absolute row position.
6) **Row-Order Preservation**: For any two predicted rows, their relative order and the relative order of their matching ground truth rows must be the same.
7) **Column-Permutation Invariance**: The metric must be invariant to column shuffling.

Attribute #1 is important because we do not often need both the cell content and location information for downstream tasks. For example, if we only want to apply business logic to the extracted data, then we only need to maximize the accuracy of cell content extraction. But if there are humans processing the output of the BDIE tool, then the cell location information becomes more important because this is what the humans could use to visually verify the extracted data.

Attribute #2 and #3 are important so that we can have an F1-score-like metric that penalizes both over-extraction and under-extraction of cells. Attribute #4 is so the metric can be extended to support multiple similarity measures that are more appropriate for downstream tasks.

Attribute #5 is relevant in cases where there are extra or missing rows in the predictions. We only want to penalize such rows and not the rows with matching ground truth rows. Attribute #6 is important because there are documents where the order of the rows matter. For example, dense financial and medical documents where the transactions are implicitly ordered by time.

And finally, Attribute #7 is an *ideal* attribute because the order of the columns do not often matter when applying business logic to the line items. Downstream programs often then save the results (and the extracted data) to spreadsheets or SQL-like databases where the order of the columns do not matter. Without this constraint, then the task reverts back to Table Structure Recognition which is a different task altogether.

A. Limitations of Current Line Item Recognition Metrics

Because of their use of Maximum-Weight Bipartite Matching-based algorithms, both ANLS* and the DocILE metrics do not satisfy attribute #6. Furthermore, ANLS* currently only supports cell content recognition. The DocILE metric, on the other hand, supports both cell content and cell location recognition but does not isolate the two. Therefore, it doesn't satisfy attribute #1. This makes it impossible to use for cell content recognition only or cell location recognition only tasks. GriTS satisfies all of the attributes above except for attribute #7.

For the rest of the section, we describe a new metric that satisfies all of the attributes above which we call General Line Items Recognition Metric (GLIRM). This metric can both be viewed as (A) an extension of ANLS* and DocILE so they would satisfy attributes #1 and #6, and (B) a relaxation of GriTS, so it would satisfy attribute #7.

B. Similarity Matching Score

As per attributes and #1 and #4, we will use $f(c_p, c_t)$ to denote the similarity measure between a predicted cell, c_p, and a ground truth cell, c_t. f can be any similarity measure appropriate for the downstream task. Exact match for

product reference numbers, Levenshtein Distance for product descriptions, Intersection-over-Union for bounding boxes, etc. We will use use $g_f(row_p, row_t)$ as the sum of the similarity scores of the corresponding cells in the predicted row, row_p, and the ground truth row, row_t.

To make the metric F1-score-like, we have to constrain f to be between 0 and 1: $0 \leq f(c_p, c_t) \leq 1$, for all c_p, c_t.

C. Row Matching

Let's denote R_p and R_t as the sequence of rows in the predicted and ground truth line items, respectively. We then denote R'_p and R'_t as *subsequences* of rows in R_p and R_t, respectively. The goal is to find equal-length subsequences $\tilde{R}_p$ and $\tilde{R}_t$ such that the sum of the similarity scores of the corresponding cells is maximized:

$$\tilde{R}_p, \tilde{R}_t = \underset{R'_p|R_p, R'_t|R_t}{\arg\max} \sum_i g_f(R'_p[i], R'_t[i]) \tag{1}$$

Because we concern ourselves with *subsequences* instead of subsets of rows, it is more appropriate to use a Levenshtein Distance-like algorithm [36] to find $\tilde{R}_p, \tilde{R}_t$ rather than a Maximum-Weight Bipartite Matching-based algorithm as in ANLS* and DocILE. This penalizes swapped or shuffled rows in the predictions.

This approach is similar to the one used in the GriTS metric, but in one dimension instead of two, and does not enforce column-order preservation.

D. General Line Items Recognition Metric Metric

To define the General Line Items Recognition Metric (GLIRM), we first define the GLIRM precision and recall scores as follows:

$$\text{GLIRM-Precision}_f(R_p, R_t) = \frac{\sum_i g_f(\tilde{R}_p[i], \tilde{R}_t[i])}{|R_t|} \tag{2}$$

$$\text{GLIRM-Recall}_f(R_p, R_t) = \frac{\sum_i g_f(\tilde{R}_p[i], \tilde{R}_t[i])}{|R_p|} \tag{3}$$

The F1-score-like GLIRM metric then is,

$$\text{GLIRM-F1}_f(R_p, R_t) = \frac{2 \times \sum_i g_f(\tilde{R}_p[i], \tilde{R}_t[i])}{|R_p| + |R_t|} \tag{4}$$

In practice or if humans are reviewing the output of the system, recall is often more important than the precision. This is because it takes more time to look for and box-in missing cells than to verify the correctness of the extracted cells. Thus, we can define the GLIRM metric as:

$$\text{GLIRM-F1}_{\beta f}(R_p, R_t) = \frac{(1+\beta^2) \times \sum_i g_f(\tilde{R}_p[i], \tilde{R}_t[i])}{\beta^2 \times |R_p| + |R_t|} \tag{5}$$

where β is a hyperparameter that controls the importance of the recall over the precision. If $\beta = 1$, then the metric is the same as GLIRM-F1$_f$.

V. Experiments

We used the DocILE dataset for benchmarking [11]. This dataset is a large-scale research benchmark for machine learning evaluation of Key Information Extraction (KIE) and Line Item Recognition (LIR) from semi-structured business documents such as invoices, orders, etc. At the time of writing this paper, this is the only dataset specifically constructed for benchmarking both KIE and LIR tasks.

We ablated three components of RASG: Retrieval Augmented Generation, Supervised Finetuning, and Structured Prompting. That is, we ran experiments on all combinations of these components. We did not include Structured Generation in the ablation benchmarks because it is a necessary component for BDIE. Without Structured Generation, we will not be able to guarantee that the output of the model is interpretable by downstream systems. We also ran experiments with two types of models: a commercially-available LLM, GPT-3.5, and an open-source LLM, Hermes 2 Pro - Mistral 7B.

We used OpenAI's Finetuning API for finetuning GPT-3.5 while we used QLoRA [37] to finetune Hermes 2 Pro - Mistral 7B on a single 80GB A100 GPU. For the retrieval mechanism, we measured the "similarity" between pages using the manhattan distance of their wavelet hashes [38]. We, however, finetuned the models for only one epoch and only used one-shot retrieval instead of many-shot retrieval primarily due to costs and token limits. Business documents are often dense, and a context window containing multiple documents quickly becomes too large for current language models to work with.

Table 1 compares the performance of LLMs with RASG on the KIE and LIR tasks, respectively, against strong, multi-modal LayoutLMv3 and Roberta + DETR baselines as described by [12] [13] [39] [40]. Analysis on the provided repository for the Docile Challenge shows that both LayoutLMv3 and Roberta baselines also employ simple Structured Prompting where they preprocess the prompt to roughly follow the natural reading order of the page, but the preprocessing is not as complex as in [16]. We then fitted a linear model to determine the contribution of each component to the overall performance of the model. Results are shown in Table 2.

The minimal resources needed to beat the baselines on the KIE task are either GPT-3.5 + 1-Shot Retrieval or Hermes 2 Pro + full RASG if one is required to run inference using open source components. For the LIR task, GPT-3.5 + 1-Shot Retrieval + Structured Prompting is sufficient to beat the baselines.

Finally, we measured the median table-level Information Coverage Score [41] for the bounding box backcalculation heuristic. The best baseline, Roberta + finetuned DETR, achieves 92.93% while GPT-3.5 + RASG and Hermes 2 Pro + RASG achieves 87.79% and 85.02%, respectively.

VI. Discussion

Our model performance and ablation results demonstrate a few conclusions. Firstly, for Key-Information Extraction

TABLE I
BENCHMARKS ON KEY INFORMATION EXTRACTION (KIE) AND LINE ITEMS RECOGNITION (LIR) ON THE VALIDATION SET OF THE DOCILE DATASET

Model	(KIE) **F1 Score**	(LIR) **GLIRM-F1 Score**
LayoutLMv3	63.95%	70.12%
LayoutLMv3 + Pretraining on synthetic data	**65.28%**	71.02%
Roberta	65.88%	70.44%
Roberta + Pretraining on synthetic data	65.97%	**73.85%**
Roberta + DETR finetuned on line items prediction	...	49.56%
Roberta + DETR finetuned on table prediction	...	72.73%
Hermes 2 Pro	16.44%	7.06%
Hermes 2 Pro + Structured Prompting *only*	16.62%	11.02%
Hermes 2 Pro + Supervised Finetuning *only*	29.68%	11.74%
Hermes 2 Pro + 1-Shot Retrieval *only*	48.42%	47.35%
Hermes 2 Pro (Full RASG)	**73.41%**	*69.44%*
GPT-3.5	26.80%	23.03%
GPT-3.5 + Structured Prompting *only*	29.30%	38.25%
GPT-3.5 + Supervised Finetuning *only*	62.40%	48.25%
GPT-3.5 + 1-Shot Retrieval *only*	**66.78%**	60.01%
GPT-3.5 (Full RASG)	**75.40%**	**79.81%**

TABLE II
ABLATION BENCHMARKS ON KEY INFORMATION EXTRACTION (KIE) AND LINE ITEMS RECOGNITION (LIR) ON THE VALIDATION SET OF THE DOCILE DATASET

Model	(KIE) **F1 Score**	(LIR) **GLIRM-F1 Score**
Baseline LLM: **GPT-3.5**	34.17%	28.31%
+ 1-Shot Retrieval	+ 22.08%	+ 20.67%
+ Supervised Finetuning	+ 22.31%	+ 17.73%
+ Structured Prompting	+ 4.96%	+ 19.42%
Baseline LLM: **Hermes 2 Pro - Mistral 7B**	13.55%	4.69%
+ 1-Shot Retrieval	+ 36.87%	+ 40.55%
+ Supervised Finetuning	+ 17.71%	+ 13.53%
+ Structured Prompting	+ 0.63%	+ 10.30%

(KIE), prompt engineering only provides marginal gains compared to augmenting the model with a retrieval mechanism and/or finetuning it on the target dataset. Perhaps this is because the structure of the document does not matter much for KIE. For Line Items Recognition (LIR), prompt engineering is as important as retrieval mechanisms and finetuning. Most interestingly, properly tuned and augmented LLMs, even the baseline Roberta model, can beat the finetuned Multi-Modal model LayoutLMv3. And lastly, our bounding box backcalculation heuristic is only slightly worse than the best baseline at table detection despite not being optimized directly for the task. These results support recent claims that vision + language models are "effectively blind" [42].

For teams working in the Business Document Information space in the future, our recommendation is to start with off-the-shelf LLMs that support structured generation. Then, implement a retrieval mechanism. If the performance is still insufficient, consider supervised finetuning. For LIR, we recommend starting with structured prompting first, then finetuning. Finally, from a dataset perspective, Line Item Recognition is still overlooked. Most publicly-available datasets are either for Table Structure Recognition or Key-Information Extraction.

VII. CONCLUSION

This paper (1) shows that augmenting LLMs with Retrieval Augmented Structured Generation (RASG) can make them even outperform multi-modal models without RASG, (2) introduces a novel metric for Line Item Recognition, GLIRM, in the context of BDIE, and (3) provides a new heuristic algorithm for Line Item Recognition that can back-calculate bounding boxes without the need for vision encoders. We demonstrate that RASG, defined as a combination of Supervised Finetuning, Structured Generation, Retrieval Augmented Generation, and Structured Prompting, is a feasible way for teams to achieve state of the art BDIE results with off-the-shelf LLMs. Additionally, we show that while Retrieval Augmented Generation crucial for both Key-Information Extraction and Line Items Recognition, Structured Prompting is often unhelpful for Key-Information Extraction.

Overall, BDIE is an exciting and practical application of LLMs. As these models improve in the coming years, there are many exciting directions for BDIE research, including multi-document retrieval and improved supervised fine-tuning and prompting engineering. The impacts of this technology to the flow of document-based information will be significant to all organizations that use documentation in any form in the future.

References

[1] R. Avyodri, S. Lukas, and H. Tjahyadi, "Optical Character Recognition (OCR) for text recognition and its post-processing method: A literature review," *2022 1st International Conference on Technology Innovation and Its Applications (ICTIIA)*, pp. 1–6, 2022. [Online]. Available: https://api.semanticscholar.org/CorpusID:253533357.

[2] X. Zhong, E. ShafieiBavani, and A. Jimeno Yepes, "Image-based table recognition: Data, model, and evaluation," in *Lecture Notes in Computer Science*. Springer International Publishing, 2020, pp. 564–580, ISBN: 9783030585891. DOI: 10.1007/978-3-030-58589-1_34. [Online]. Available: http://dx.doi.org/10.1007/978-3-030-58589-1_34.

[3] A. Nassar, N. Livathinos, M. Lysak, and P. Staar, "TableFormer: Table structure understanding with Transformers," in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, IEEE, Jun. 2022. DOI: 10.1109/cvpr52688.2022.00457. [Online]. Available: http://dx.doi.org/10.1109/CVPR52688.2022.00457.

[4] Y. Huang, N. Lu, D. Chen, *et al.*, "Improving table structure recognition with visual-alignment sequential coordinate modeling," in *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, IEEE, Jun. 2023. DOI: 10.1109/cvpr52729.2023.01071. [Online]. Available: http://dx.doi.org/10.1109/CVPR52729.2023.01071.

[5] M. Mathew, D. Karatzas, and C. V. Jawahar, "DocVQA: A dataset for VQA on document images," in *2021 IEEE Winter Conference on Applications of Computer Vision (WACV)*, IEEE, Jan. 2021. DOI: 10.1109/wacv48630.2021.00225. [Online]. Available: http://dx.doi.org/10.1109/WACV48630.2021.00225.

[6] J. Hammerton, "Named Entity Recognition with Long Short-Term Memory," in *Proceedings of the Seventh Conference on Natural Language Learning at HLT-NAACL 2003*, 2003, pp. 172–175. [Online]. Available: https://aclanthology.org/W03-0426.

[7] M. Skalický, Š. Šimsa, M. Uřičář, and M. Šulc, *Business document information extraction: Towards practical benchmarks*, 2023. arXiv: 2206.11229 [cs.IR].

[8] G. Jaume, H. Kemal Ekenel, and J.-P. Thiran, "FUNSD: A dataset for form understanding in noisy scanned documents," in *2019 International Conference on Document Analysis and Recognition Workshops (ICDARW)*, IEEE, Sep. 2019. DOI: 10.1109/icdarw.2019.10029. [Online]. Available: http://dx.doi.org/10.1109/ICDARW.2019.10029.

[9] S. Park, S. Shin, B. Lee, *et al.*, "CORD: A consolidated receipt dataset for post-OCR parsing," in *Workshop on Document Intelligence at NeurIPS 2019*, 2019. [Online]. Available: https://openreview.net/forum?id=SJl3z659UH.

[10] Z. Huang, K. Chen, J. He, *et al.*, "ICDAR2019 Competition on scanned receipt OCR and information extraction," in *2019 International Conference on Document Analysis and Recognition (ICDAR)*, IEEE, Sep. 2019. DOI: 10.1109/icdar.2019.00244. [Online]. Available: http://dx.doi.org/10.1109/ICDAR.2019.00244.

[11] Š. Šimsa, M. Šulc, M. Uřičář, *et al.*, *DocILE benchmark for document information localization and extraction*, 2023. arXiv: 2302.05658 [cs.CL].

[12] Š. Šimsa, M. Uřičář, M. Šulc, *et al.*, "Overview of DocILE 2023: Document information localization and extraction," in *Experimental IR Meets Multilinguality, Multimodality, and Interaction*, Cham: Springer Nature Switzerland, 2023, pp. 276–293, ISBN: 978-3-031-42448-9.

[13] Y. Huang, T. Lv, L. Cui, Y. Lu, and F. Wei, "LayoutLMv3: Pre-training for Document AI with unified text and image masking," in *Proceedings of the 30th ACM International Conference on Multimedia*, ser. MM '22, ACM, Oct. 2022. DOI: 10.1145/3503161.3548112. [Online]. Available: http://dx.doi.org/10.1145/3503161.3548112.

[14] D. Wang, N. Raman, M. Sibue, *et al.*, *DocLLM: A layout-aware generative language model for multimodal document understanding*, 2023. arXiv: 2401.00908 [cs.CL].

[15] T. Dinh, Y. Zeng, R. Zhang, *et al.*, *LIFT: Language-interfaced fine-tuning for non-language machine learning tasks*, 2022. arXiv: 2206.06565 [cs.LG].

[16] W. Wang, Y. Li, Y. Ou, and Y. Zhang, *Layout and task aware instruction prompt for zero-shot document image question answering*, 2023. arXiv: 2306.00526 [cs.CL].

[17] S. Small and L. Medsker, "Review of information extraction technologies and applications," *Neural Computing and Applications*, Dec. 2013. DOI: 10.1007/s00521-013-1516-6.

[18] H. Yang, C. A. Aguirre, M. F. De La Torre, *et al.*, "Pipelines for procedural information extraction from scientific literature: Towards recipes using machine learning and data science," in *2019 International Conference on Document Analysis and Recognition Workshops (ICDARW)*, vol. 2, 2019, pp. 41–46. DOI: 10.1109/ICDARW.2019.10037.

[19] K. Toutanova, D. Klein, C. D. M. Manning, and Y. Singer, "Feature-rich part-of-speech tagging with a Cyclic Dependency Network," in *Proceedings of the 2003 Human Language Technology Conference of the North American Chapter of the Association for Computational Linguistics*, 2003, pp. 252–259. [Online]. Available: https://aclanthology.org/N03-1033.

[20] V. Perot, K. Kang, F. Luisier, *et al.*, *LMDX: Language model-based document information extraction and localization*, 2023. arXiv: 2309.10952 [cs.CL].

[21] B. Smock, R. Pesala, and R. Abraham, *GriTS: Grid table similarity metric for table structure recognition*, 2023. arXiv: 2203.12555 [cs.LG].

[22] D. Peer, P. Schöpf, V. Nebendahl, A. Rietzler, and S. Stabinger, *ANLS* – a universal document processing metric for generative large language models*, 2024. arXiv: 2402.03848 [cs.CL].

[23] H. W. Kuhn, "The Hungarian method for the Assignment Problem," *Naval Research Logistics Quarterly*, vol. 2, no. 1-2, pp. 83–97, 1955.

[24] O. Ram, Y. Levine, I. Dalmedigos, *et al.*, "In-context retrieval-augmented language models," *Transactions of the Association for Computational Linguistics*, vol. 11, pp. 1316–1331, 2023, ISSN: 2307-387X. DOI: 10.1162/tacl_a_00605. [Online]. Available: http://dx.doi.org/10.1162/tacl_a_00605.

[25] P. Lewis, E. Perez, A. Piktus, *et al.*, *Retrieval-augmented generation for knowledge-intensive NLP tasks*, 2021. arXiv: 2005.11401 [cs.CL].

[26] T. Schick, J. Dwivedi-Yu, R. Dessì, *et al.*, *Toolformer: Language models can teach themselves to use tools*, 2023. arXiv: 2302.04761 [cs.CL].

[27] J. Liu, *Instructor*, https://python.useinstructor.com/ [Accessed: 2024-03-08], 2023.

[28] B. T. Willard and R. Louf, *Efficient guided generation for large language models*, 2023. arXiv: 2307.09702 [cs.CL].

[29] M. Lee, J. Min, W. Lee, and Y. Lee, *Structured language generation model for robust structure prediction*, 2024. arXiv: 2402.08971 [cs.CL].

[30] Q. Zhang, J. Dong, H. Chen, *et al.*, *Structure guided large language model for SQL generation*, 2024. arXiv: 2402.13284 [cs.DB].

[31] interstellarninja, Teknium, theemozilla, karan4d, and huemin_art, *Hermes 2 Pro - Mistral 7B*, https://huggingface.co/NousResearch/Hermes-2-Pro-Mistral-7B [Accessed: 2024-03-08], 2024.

[32] OpenAI, *GPT-3.5 Documentation*, https://platform.openai.com/docs/models/gpt-3-5-turbo [Accessed: 2024-03-08], 2023.

[33] J. Tang and T. Addair, *LoRAX + Outlines: Better JSON extraction with structured generation and LoRA*, https://predibase.com/blog/lorax-outlines-better-json-extraction-with-structured-generation-and-lora [Accessed: 2024-03-08], 2024.

[34] R. Bellman, *Dynamic Programming*. Stanford University Press, 2023.

[35] Algorithms for Competitive Programming Team, A. Sastry, and S. Zahin, *Divide and Conquer DP*, https://cp-algorithms.com/dynamic_programming/divide-and-conquer-dp.html [Accessed: 2024-03-08], 2023.

[36] R. A. Wagner and M. J. Fischer, "The string-to-string correction problem," *Journal of the ACM (JACM)*, vol. 21, no. 1, pp. 168–173, 1974. DOI: 10.1145/321796.321811.

[37] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, *QLoRA: Efficient finetuning of quantized LLMs*, 2023. arXiv: 2305.14314 [cs.LG].

[38] S. P. Singh and G. Bhatnagar, "A robust image hashing based on discrete wavelet transform," in *2017 IEEE International Conference on Signal and Image Processing Applications (ICSIPA)*, 2017, pp. 440–444. DOI: 10.1109/ICSIPA.2017.8120651.

[39] Y. Liu, M. Ott, N. Goyal, *et al.*, *RoBERTa: A robustly optimized BERT pretraining approach*, 2019. arXiv: 1907.11692 [cs.CL].

[40] N. Carion, F. Massa, G. Synnaeve, *et al.*, "End-to-end object detection with Transformers," in *Lecture Notes in Computer Science*. Springer International Publishing, 2020, pp. 213–229, ISBN: 9783030584528. DOI: 10.1007/978-3-030-58452-8_13. [Online]. Available: http://dx.doi.org/10.1007/978-3-030-58452-8_13.

[41] B. Xiao, M. Simsek, B. Kantarci, and A. A. Alkheir, "Table detection for visually rich document images," *Knowledge-Based Systems*, vol. 282, p. 111 080, 2023, ISSN: 0950-7051. DOI: 10.1016/j.knosys.2023.111080. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0950705123008304.

[42] A. Majumdar, A. Ajay, X. Zhang, *et al.*, "OpenEQA: Embodied question answering in the era of foundation models," in *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

Appendix

A. Ablation Benchmarks

TABLE III
ABLATION BENCHMARK RAW RESULTS FOR KEY-INFORMATION EXTRACTION (KIE) WITH GPT-3.5

1-Shot Retrieval	Supervised Fine-Tuning	Structured Prompting	F1 Score
✓	✓	✓	**75.40%**
✗	✓	✓	**72.72%**
✓	✓	✗	**69.48%**
✓	✗	✓	**67.87%**
✓	✗	✗	**66.78%**
✗	✓	✗	62.40%
✗	✗	✓	29.30%
✗	✗	✗	26.80%

TABLE IV
ABLATION BENCHMARK RAW RESULTS FOR KEY-INFORMATION EXTRACTION (KIE) WITH HERMES 2 PRO

1-Shot Retrieval	Supervised Fine-Tuning	Structured Prompting	F1 Score
✓	✓	✓	**73.41%**
✓	✓	✗	68.82%
✓	✗	✗	48.42%
✓	✗	✓	47.71%
✗	✓	✗	29.68%
✗	✓	✓	28.13%
✗	✗	✓	16.62%
✗	✗	✗	16.44%

TABLE V
ABLATION BENCHMARK RAW RESULTS FOR LINE ITEMS RECOGNITION (LIR) WITH GPT-3.5

1-Shot Retrieval	Supervised Fine-Tuning	Structured Prompting	GLIRM-F1 Score
✓	✓	✓	**79.81%**
✗	✓	✓	**77.52%**
✓	✗	✓	**72.12%**
✓	✗	✗	58.26%
✓	✓	✗	48.72%
✗	✓	✗	48.72%
✗	✗	✓	38.25%
✗	✗	✗	23.03%

TABLE VI
ABLATION BENCHMARK RAW RESULTS FOR LINE ITEMS RECOGNITION (LIR) WITH HERMES 2 PRO

1-Shot Retrieval	Supervised Fine-Tuning	Structured Prompting	GLIRM-F1 Score
✓	✓	✓	69.44%
✓	✓	✗	56.80%
✓	✗	✓	55.02%
✓	✗	✗	47.35%
✗	✓	✓	36.61%
✗	✓	✗	11.74%
✗	✗	✗	11.02%
✗	✗	✓	7.06%